2016 中国互联网安全大会
China Internet Security Conference

协同联动 共建安全+命运共同体

QEMU漏洞之殇

李强

奇虎360信息安全部
云安全研究员

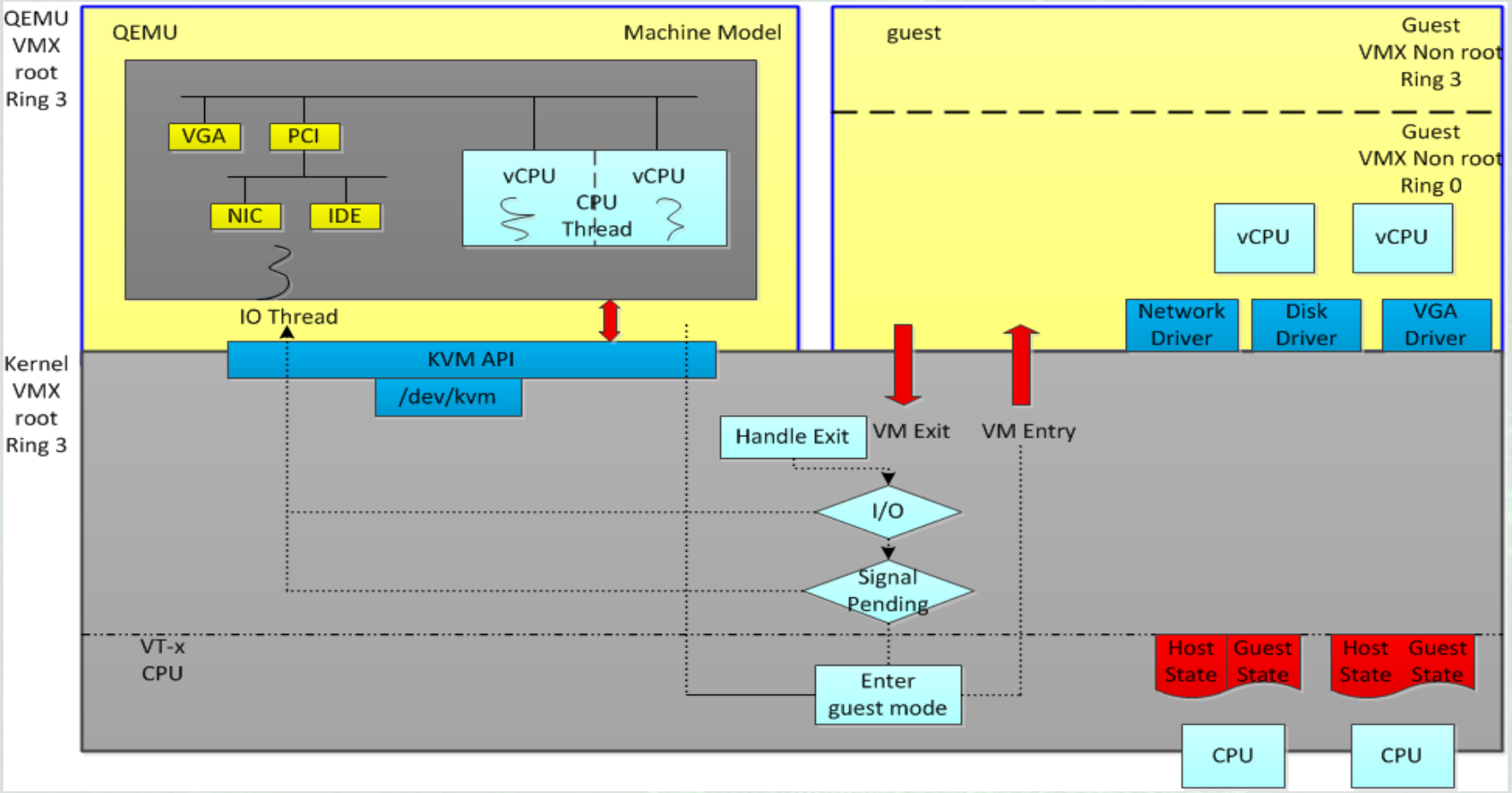
关于我

- 奇虎360信息安全部
- 漏洞挖掘与利用
- QEMU安全研究
  -- 目前发现30多个漏洞，已获得14个CVE，剩余仍在处理

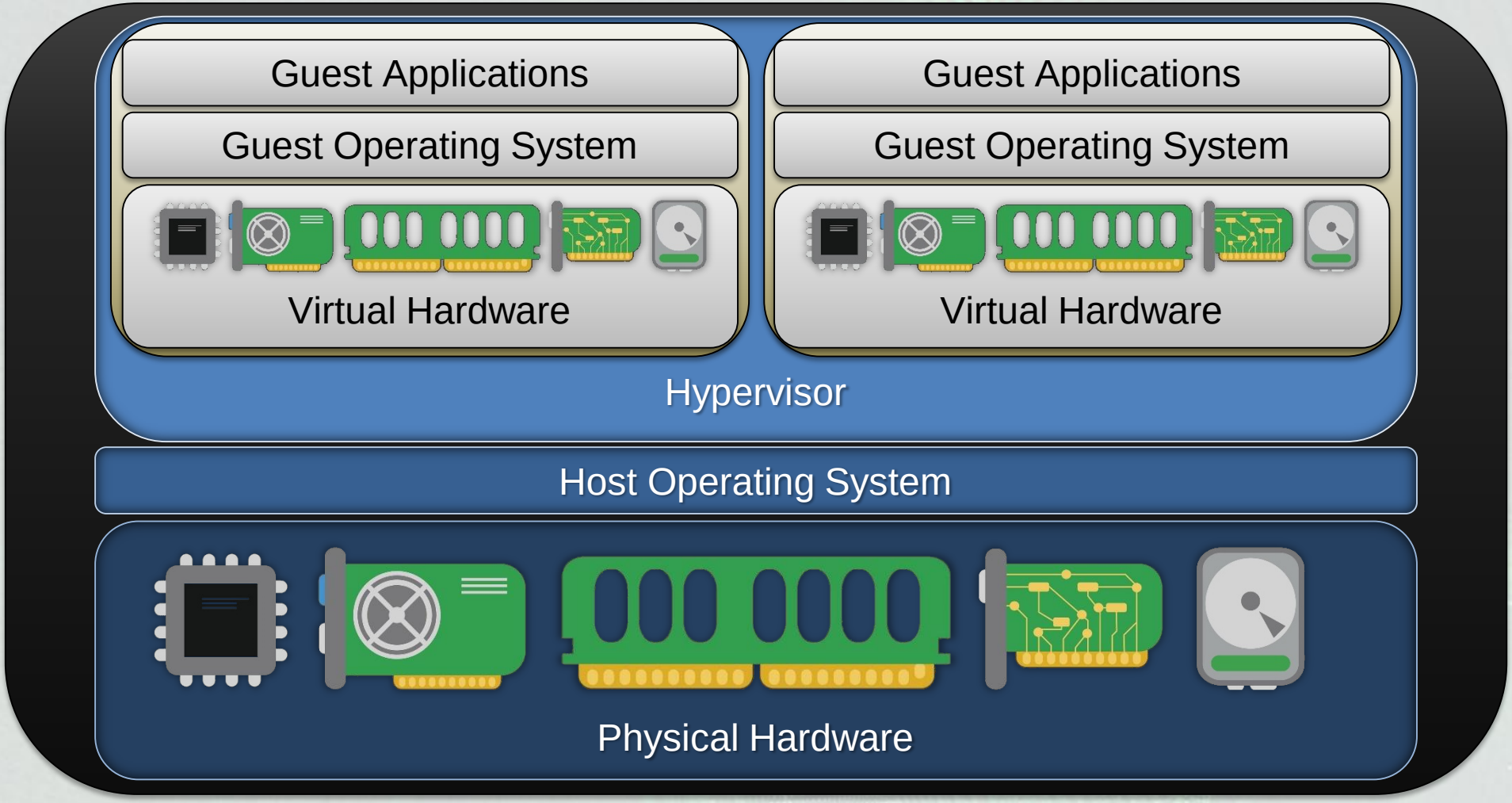
目录

QEMU简介

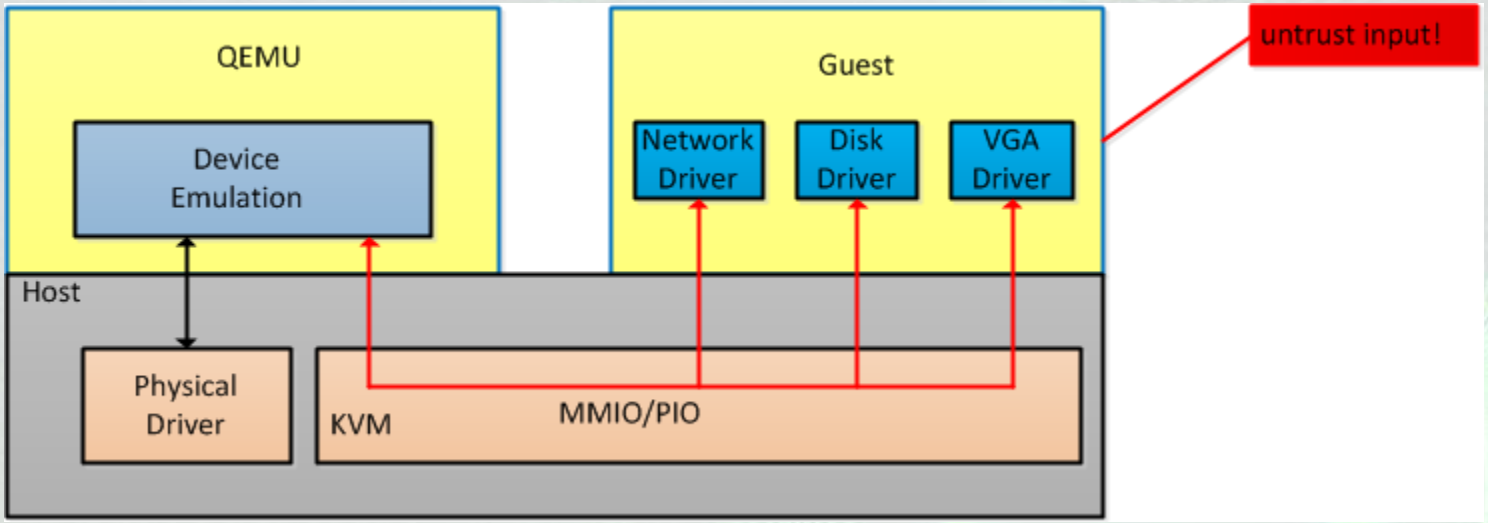
QEMU整体架构


QEMU
VMX
root
Ring 3
QEMU
Machine Model
VGA
PCI
NIC
IDE
vCPU
vCPU
CPU
Thread
IO Thread
guest
Guest
VMX Non root
Ring 3
Guest
VMX Non root
Ring 0
vCPU
vCPU
Network
Driver
Disk
Driver
VGA
Driver
Kernel
VMX
root
Ring 3
KVM API
/dev/kvm
Handle Exit
VM Exit
VM Entry
I/O
Signal
Pending
VT-x
CPU
Enter
guest mode
Host
State
Guest
State
Host
State
Guest
State
CPU
CPU

QEMU设备模拟

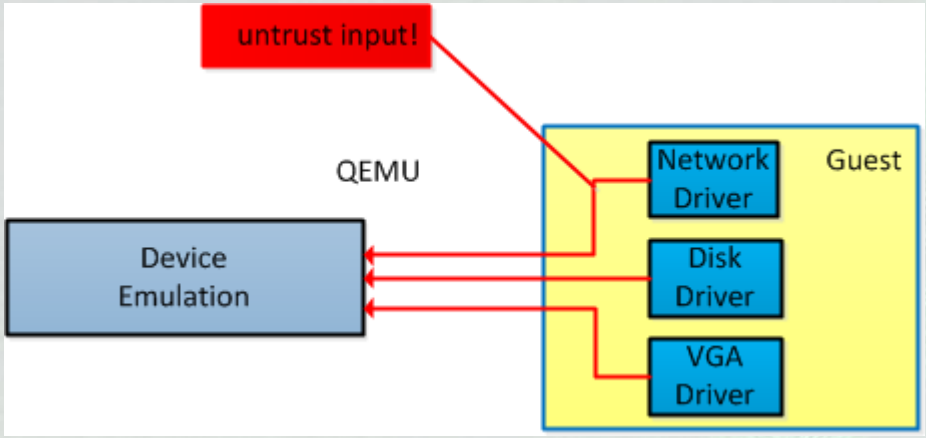

Guest Applications
Guest Operating System
Virtual Hardware
Guest Applications
Guest Operating System
Virtual Hardware
Hypervisor
Host Operating System
Physical Hardware

QEMU设备模拟



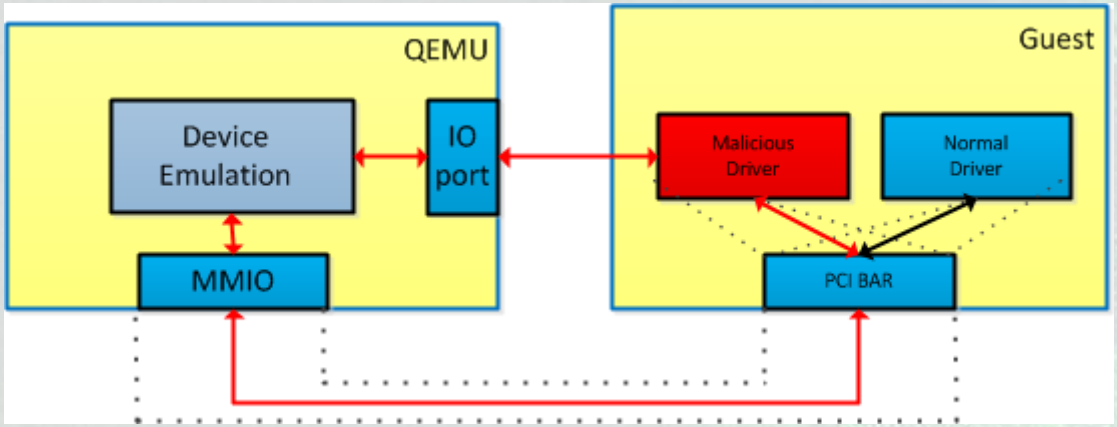
- 虚拟机中操作系统期待真实的硬件
- QEMU模拟各种各样的硬件
- 网卡，VGA，磁盘等

QEMU设备模拟



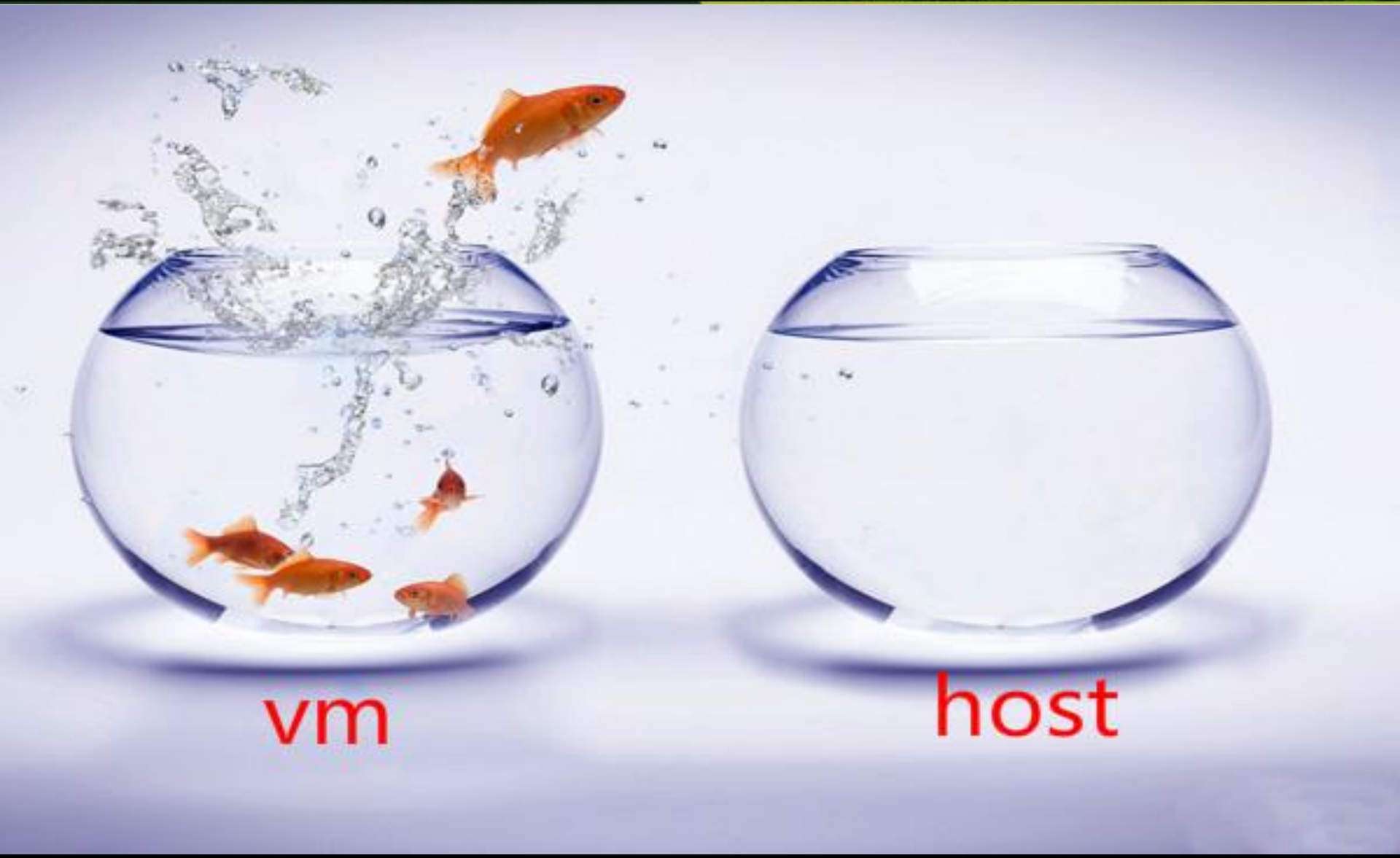
- KVM做转发
- 虚拟机->QEMU的数据流

QEMU设备模拟

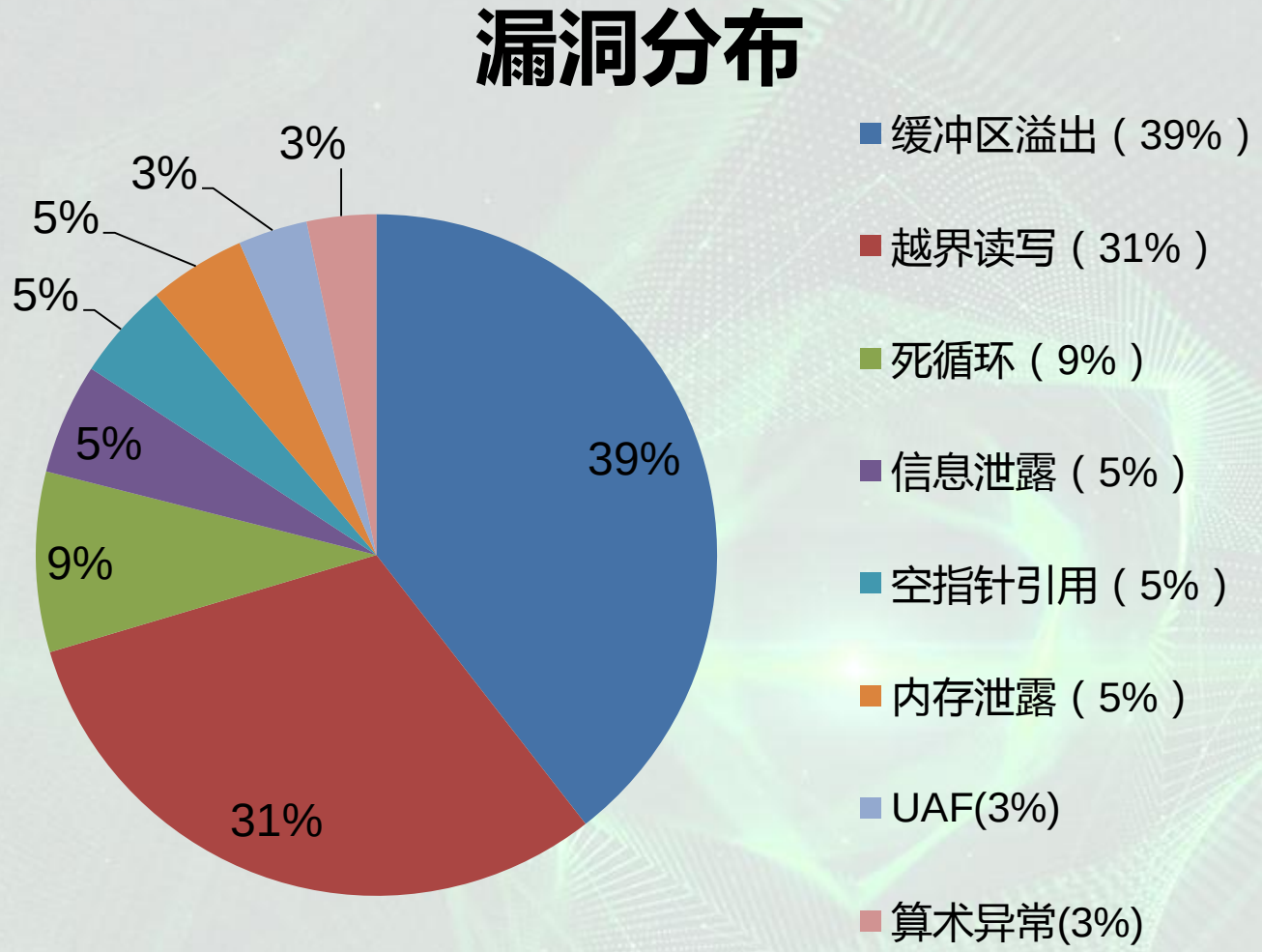


- QEMU映射一段内存作为模拟设备的BAR->MMIO
- QEMU为模拟设备注册端口->PIO
- 正常的设备驱动通过IO port和MMIO驱动设备
- 恶意驱动也可以写设备的IO port和MMIO

QEMU漏洞分析



QEMU漏洞分析


漏洞分布
39%
31%
9%
5%
5%
3%
3%
缓冲区溢出（39%）
越界读写（31%）
死循环（9%）
信息泄露（5%）
空指针引用（5%）
内存泄露（5%）
UAF(3%)
算术异常(3%)

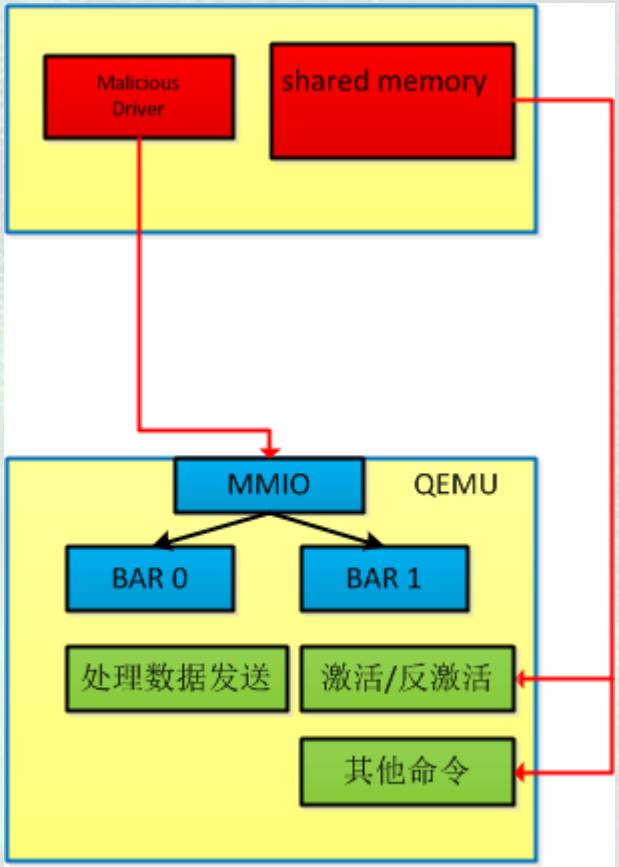
vmxnet3漏洞分析

- QEMU漏洞种类繁多
- 一个设备一般也就2、3种漏洞
- vmxnet3有6种不同类型的漏洞(本人独立发现其中的5个），包括内存泄露，UAF，空指针引用，死循环，信息泄露，缓冲区溢出

QEMU漏洞分析

vmxnet3设备模拟

- driver提供一块shared memory的物理地址给vmxnet3网卡
- vmxnet3网卡从shared memory取数据，执行各种命令



QEMU漏洞分析

内存泄露

- 漏洞发现：内存泄露是QEMU中很常见的bug
- 漏洞成因：激活设备时未对当前设备的状态进行判断，能够多次激活设备
- 漏洞危害：内存泄露，耗尽宿主机内存

```
static void vmxnet3_activate_device(VMXNET3State *s)
{
    /* Verify if device is active */
+   if (s->device_active) {
+       VMW_CFPRN("Vmxnet3 device is active");
+       return;
+   }
    ...
    /* Preallocate TX packet wrapper */
    VMW_CFPRN("Max TX fragments is %u", s->max_tx_frags);
    vmxnet_tx_pkt_init(&s->tx_pkt, s->max_tx_frags, s->peer_has_vhdr);
    vmxnet_rx_pkt_init(&s->rx_pkt, s->peer_has_vhdr);
    ...
}

void vmxnet_tx_pkt_init(struct VmxnetTxPkt **pkt, uint32_t max_frags,
    bool has_virt_hdr)
{
    struct VmxnetTxPkt *p = g_malloc0(sizeof *p);

    p->vec = g_malloc((sizeof *p->vec) *
        (max_frags + VMXNET_TX_PKT_PL_START_FRAG));

    p->raw = g_malloc((sizeof *p->raw) * max_frags);
    ...
}
```

QEMU漏洞分析

UAF

- 漏洞发现：意外发现，将vmxnet3反激活之后，发现进程崩溃
- 漏洞成因：在将设备反激活之后，相关数据结构已经free掉了，但是依然可以写bar 0空间，从而使用释放掉的数据结构
- 漏洞危害：代码执行，QEMU进程崩溃

```
static void vmxnet3_deactivate_device(VMXNET3State *s)
{
    if (s->device_active) {
        ...
        vmxnet_tx_pkt_uninit(s->tx_pkt);
        ...
    }
}

void vmxnet_tx_pkt_uninit(struct VmxnetTxPkt *pkt)
{
    if (pkt) {
        ...
        g_free(pkt);
    }
}
```

```
static void
vmxnet3_io_bar0_write(void *opaque, hwaddr addr,
                      uint64_t val, unsigned size)
{
    +    if (!s->device_active) {
    +        return;
    +    }
    +
    vmxnet3_process_tx_queue(s, tx_queue_idx);
}

static void vmxnet3_process_tx_queue(VMXNET3State *s, int qidx)
{
    vmxnet_tx_pkt_add_raw_fragment(s->tx_pkt, data_pa, data_len);
}
```

QEMU漏洞分析

空指针引用

- 漏洞发现：这里本质上是一个整数溢出漏洞
- 漏洞成因：没有对max_frags做检查，导致了g_malloc(0)，p->raw赋值为NULL
- 漏洞危害：QEMU进程崩溃

```
void vmxnet_tx_pkt_init(struct VmxnetTxPkt **pkt, uint32_t max_frags,
    bool has_virt_hdr)
{
+    assert(max_frags <= 16);
    p->vec = g_malloc((sizeof *p->vec) *
        (max_frags + VMXNET_TX_PKT_PL_START_FRAG));
    p->raw = g_malloc((sizeof *p->raw) * max_frags);

}

bool vmxnet_tx_pkt_add_raw_fragment(struct VmxnetTxPkt *pkt, hwaddr pa,
    size_t len)
{
    ventry = &pkt->raw[pkt->raw_frags];
    mapped_len = len;

    ventry->iov_base = cpu_physical_memory_map(pa, &mapped_len, false);
    ventry->iov_len = mapped_len;
}
```

QEMU漏洞分析

死循环

- 漏洞发现：典型的QEMU漏洞之一，循环处理网络数据包、SCSI和USB主控处理请求帧都经常存在这类问题
- 漏洞成因：未考虑fragment_len=0的情况，会导致more_frags不变，循环条件不会改变
- 漏洞危害：会导致QEMU虚拟机CPU占用率长时间保持在100%，拒绝服务

```
static bool vmxnet_tx_pkt_do_sw_fragmentation(struct VmxnetTxPkt *pkt,
    NetClientState *nc)
{

    /* Put as much data as possible and send */
    do {
        fragment_len = vmxnet_tx_pkt_fetch_fragment(pkt, &src_idx, &src_offset,
            fragment, &dst_idx);

        more_frags = (fragment_offset + fragment_len < pkt->payload_len);
        ...
        fragment_offset += fragment_len;

-    } while (more_frags);
+    } while (fragment_len && more_frags);

}
```

QEMU漏洞分析

信息泄露

- 漏洞发现：QEMU常见漏洞之一，QEMU向虚拟机写入数据
- 漏洞成因：txcq_descr没有完全初始化就写入虚拟机内存
- 漏洞危害：泄露QEMU栈上的数据，能够用于ASLR的绕过

```
struct Vmxnet3_TxCompDesc {
    u32         txdIdx:12;    /* Index of the EOP TxDesc */
    u32         ext1:20;

    __le32      ext2;
    __le32      ext3;

    u32         rsvd:24;
    u32         type:7;       /* completion type */
    u32         gen:1;        /* generation bit */
};

static void vmxnet3_complete_packet(VMXNET3State *s, int qidx, uint32_t tx_ridx)
{
    struct Vmxnet3_TxCompDesc txcq_descr;
+    memset(&txcq_descr, 0, sizeof(txcq_descr));
    txcq_descr.txdIdx = tx_ridx;
    txcq_descr.gen = vmxnet3_ring_curr_gen(&s->txq_descr[qidx].comp_ring);
    vmxnet3_ring_write_curr_cell(&s->txq_descr[qidx].comp_ring, &txcq_descr);
}
```

CVE-2016-2841

QEMU漏洞分析

堆溢出

- 漏洞发现：网卡在处理数据包的过程中，会使用数据包中的数据
- 漏洞成因：l3_hdr->iov_len从数据包中解析而来，表示IP头大小，未做大小判断，在iov_bo_buf，最后一个参数会导致整数下溢
- 漏洞危害：代码执行，QEMU进程崩溃

```
static bool vmxnet_tx_pkt_parse_headers(struct VmxnetTxPkt *pkt)
{

    switch (l3_proto) {
    case ETH_P_IP:

        l3_hdr->iov_len = IP_HDR_GET_LEN(l3_hdr->iov_base);

+       if(l3_hdr->iov_len < sizeof(struct ip_header))
+       {
+           l3_hdr->iov_len = 0;
+           return false;
+       }

        bytes_read = iov_to_buf(pkt->raw, pkt->raw_frags,
                                l2_hdr->iov_len + sizeof(struct ip_header),
                                l3_hdr->iov_base + sizeof(struct ip_header),
                                l3_hdr->iov_len - sizeof(struct ip_header));

    return true;
}
```

CVE-2016-2841
CVE-2016-2841

QEMU漏洞挖掘心得

代码审计

发现漏洞

CVE-2016-2841

QEMU漏洞挖掘心得

- 太阳底下无新事，QEMU漏洞与其他软件漏洞并没有什么区别
- 熟悉数据流
- 熟悉漏洞类型

结语

- 虚拟机逃逸相关：CVE-2016-4439
- 致谢：蔡玉光，胡智斌
- Q&A

谢　谢